Andol Saketh

12221210

SET B

1. write a recursive function sumNatural(n:int):Int in Scala to calculate the sum of the first n natural numbers

```
scala> 2.  if (n == 0) 0
           ^
       error: identifier expected but 'if' found.

scala> 3.  else n + sumNatural(n - 1)
           ^
       error: identifier expected but 'else' found.

scala> 4.}
         ^
       error: identifier expected but '}' found.

scala> def sumNatural(n:Int): Int = {
     | if(n == 0)0
     | else n + sumNatural(n - 1)
     | }
def sumNatural(n: Int): Int

scala> println(sumNatural(9))
45

scala> def sumNatural(n: Int): Int = {
     |   if (n == 0) 0
     |   else n + sumNatural(n - 1)
     | }
def sumNatural(n: Int): Int

scala> println(sumNatural(22))
253

scala> def sumNatural(n: Int): Int = {
     |   if (n == 0) 0
     |   else n + sumNatural(n - 1)
     | }
def sumNatural(n: Int): Int

scala> println(sumNatural(99))
4950

scala> def sumNatural(n: Int): Int = {
     |   if (n == 0) 0
     |   else n + sumNatural(n - 1)
     | }
def sumNatural(n: Int): Int

scala> println(sumNatural(123))
7626

scala>
```

2. You are given to RDD pf key -value pairs representing product sales in a store :
   [(“Laptop”,3) , (“Phone”,5),(“Tablet”,2),(“Laptop”,4),(“Phone”,1)]
   Write a spark program in Scala to :
   A.Use reduceBy key to calculate the total quantity sold for each product

   B.Filter out products with total sales less than 5.
   C.Display the remaining products with their total sales

```
val sales: org.apache.spark.rdd.RDD[(String, Int)] = ParallelCollectionRDD[6] at parallelize at <console>:1

scala> val totalSales = sales.reduceByKey(_ + _)
val totalSales: org.apache.spark.rdd.RDD[(String, Int)] = ShuffledRDD[7] at reduceByKey at <console>:1

scala> filteredSales.collect().foreach(println)
       ^
       error: not found: value filteredSales

scala> val filteredSales = totalSales.filter { case (_, qty) => qty < 5 }
val filteredSales: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[8] at filter at <console>:1

scala> filteredSales.collect().foreach { case (product, qty) =>
     | println(s"$product -> $qty")
     |
     |
You typed two blank lines.  Starting a new command.

scala> filteredSales.collect().foreach(println)
(Tablet,2)

scala> val sales = sc.parallelize(Seq(
     |       ("Laptop", 3),
     |       ("Phone", 5),
     |       ("Tablet", 2),
     |       ("Laptop", 4),
     |       ("Phone", 1)
     |     ))
val sales: org.apache.spark.rdd.RDD[(String, Int)] = ParallelCollectionRDD[9] at parallelize at <console>:1

scala> val filteredSales = totalSales.filter { case (_, qty) => qty < 5 }
val filteredSales: org.apache.spark.rdd.RDD[(String, Int)] = MapPartitionsRDD[10] at filter at <console>:1

scala> filteredSales.collect().foreach { case (product, qty) =>
     | println(s"$product -> $qty")
     |
     |
You typed two blank lines.  Starting a new command.

scala> filteredSales.collect().foreach(println)
(Tablet,2)

scala>
```